

从 AndroidManifest.xml 中读取 Activity 的元数据

这段代码的作用是从当前 Activity 的清单文件（AndroidManifest.xml）中读取自定义的元数据（**meta-data**），并显示在 TextView 上。咱们一步步拆开来讲，让小白也能懂～

一、代码逐行解释

先看完整代码的作用：从清单文件中读取 key 为 `weather` 的元数据，显示到 `tv_string2` 这个文本控件上。

```
// 1. 获取布局中的 TextView 控件（用来显示读取到的元数据）
TextView textView2 = findViewById(R.id.tv_string2);
// 2. 获取 PackageManager 对象（PackageManager 是管理应用信息的“大管家”）
PackageManager pm = getPackageManager();
try {
    // 3. 获取当前 Activity 的信息（包含清单文件中配置的元数据）
    // getComponentName(): 获取当前 Activity 的“组件名”（相当于身份证）
    // PackageManager.GET_META_DATA: 告诉大管家“我要获取元数据”
    ActivityInfo info = pm.getActivityInfo(getComponentName(), PackageManager.GET_META_
DATA);
    // 4. 从 ActivityInfo 中取出元数据（元数据以 Bundle 形式存储，类似键值对）
    Bundle bundle = info.metaData;
    // 5. 从 Bundle 中根据 key（这里是"weather"）取出对应的值
    String weather = bundle.getString("weather");
    // 6. 把读取到的值显示到 TextView 上
    textView2.setText(weather);
} catch (PackageManager.NameNotFoundException e) {
    // 如果找不到对应的 Activity 信息，抛出异常
    throw new RuntimeException(e);
}
```

二、关键前提：清单文件中必须配置 meta-data

代码能读到数据的前提是：在 AndroidManifest.xml 中，给当前 Activity 配置了 `meta-data`。例如：

```
<activity
    android:name=".MainActivity"
```

```
android:exported="true">
<!-- 配置元数据：key 是"weather"，value 是"晴天" -->
<meta-data
    android:name="weather" <!-- 键 (key) -->
    android:value="晴天" <!-- 值 (value) -->
/>
<!-- 其他配置（如 intent-filter） -->
</activity>
```

- `meta-data` 是 Android 清单文件中用于存储**额外配置信息**的标签，形式是“键值对”（key-value）。
- 它的作用是：让一些“配置信息”和代码分离，方便修改（比如改 `weather` 的值，直接改清单文件即可，不用动代码）。

三、核心类和方法解析

1. PackageManager

系统提供的“应用信息管理工具”，可以获取应用的 Activity、Service、权限、元数据等信息。通过 `getPackageManager()` 方法获取其实例。

2. ActivityInfo

存储单个 Activity 的详细信息（如名称、图标、元数据等）。通过 `pm.getActivityInfo(...)` 可以获取指定 Activity 的信息。

3. getComponentName()

获取当前 Activity 的“组件名”（包含包名和类名），相当于告诉 `PackageManager`：“我要获取**当前这个 Activity** 的信息”。

4. PackageManager.GET_META_DATA

一个“标志位”，告诉 `PackageManager`：“在获取 Activity 信息时，顺便把它的 meta-data 也取出来”（如果不加这个标志，`info.metaData` 会是 null）。

5. metaData

`ActivityInfo` 中的一个 `Bundle` 对象，专门存储该 Activity 在清单文件中配置的所有 meta-data 键值对。

四、扩展：meta-data 的更多用法

1. 配置到 Application 节点

meta-data 不仅能配置在 Activity 中，还能配置在 Application 节点（全局生效）：

```
<application ...>
  <meta-data
    android:name="app_version"
    android:value="1.0.0"
  />
</application>
```

读取时用 `pm.getApplicationInfo(...)`：

```
ApplicationInfo appInfo = pm.getApplicationInfo(getPackageName(),
PackageManager.GET_META_DATA);
String version = appInfo.metaData.getString("app_version");
```

2. 存储复杂数据

除了字符串，还能存储整数、布尔值等，甚至通过 `resource` 引用资源文件：

```
<meta-data
  android:name="max_count"
  android:value="100" <!-- 整数 -->
/>
<meta-data
  android:name="is_debug"
  android:value="true" <!-- 布尔值 -->
/>
<meta-data
  android:name="app_name"
  android:resource="@string/app_name" <!-- 引用字符串资源 -->
/>
```

读取时用对应方法（如 `getInt("max_count")`、`getBoolean("is_debug")`）。

3. 第三方 SDK 常用

很多 SDK（如百度地图、微信分享）会要求在清单文件中配置 meta-data 来传递密钥（AppKey）：

```
<meta-data
  android:name="com.baidu.lbsapi.API_KEY"
```

```
android:value="你的百度地图密钥"  
</>
```

SDK 内部会通过类似本文的代码读取密钥，实现初始化。

五、小白总结

这段代码的核心是**读取清单文件中为当前 Activity 配置的 meta-data**，流程是：

1. 用 `PackageManager` 拿到当前 Activity 的信息 (`ActivityInfo`) ；
2. 从 `ActivityInfo` 中取出 meta-data (`Bundle`) ；
3. 根据 key 读取值，显示到界面上。

meta-data 适合存储一些 “配置型信息”，优点是**不用改代码就能修改值**，广泛用于应用配置和第三方 SDK 集成～

（注：文档部分内容可能由 AI 生成）